Introducción a ARM Cortex-M4
Startup y Toolchain

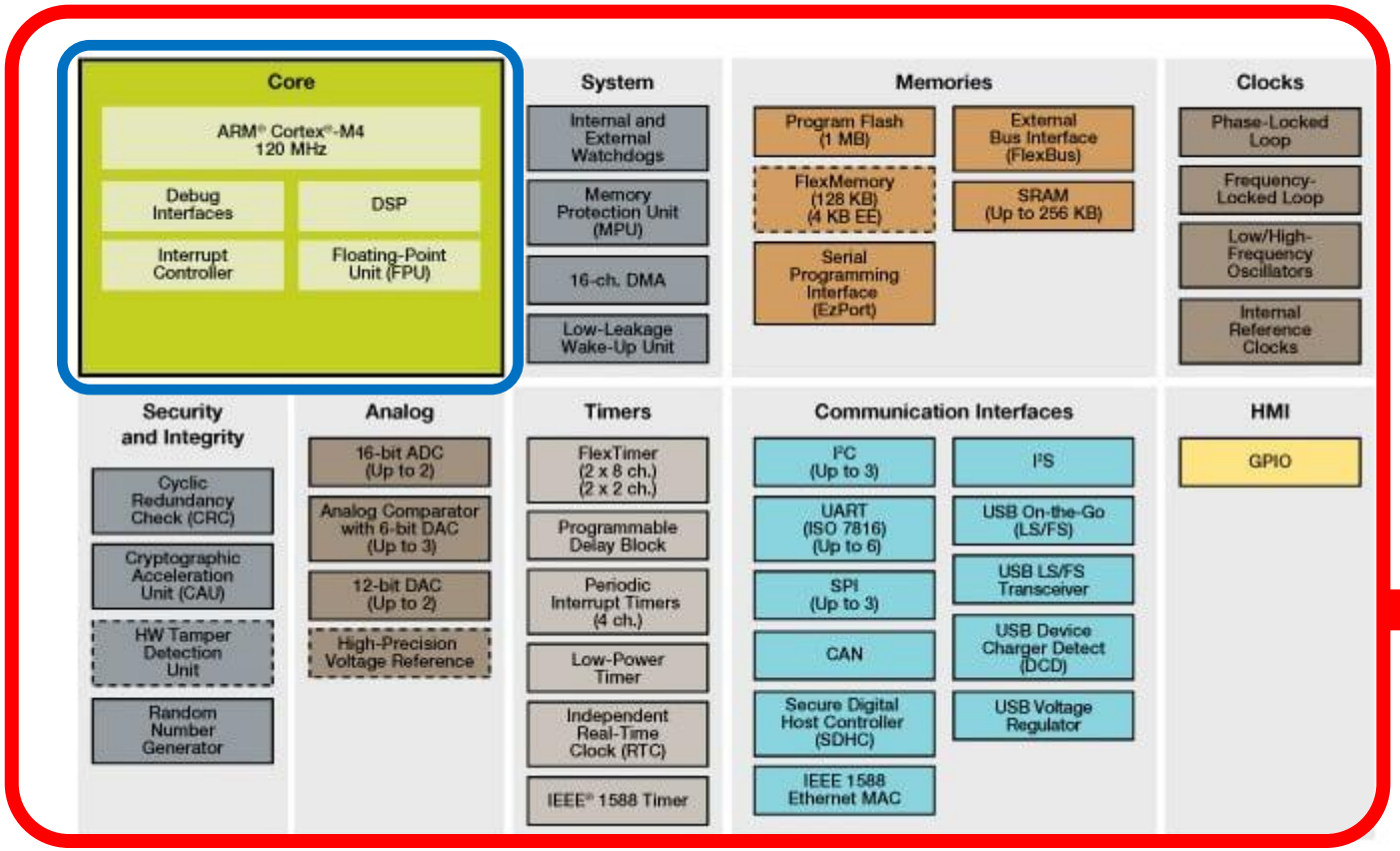
```
 5:ej1.c          ****     while (p--) {
30                        .loc 1 5 0
31 0000 03E0              b   .L2
32                 .L3:
 6:ej1.c          ****         q++;
33                        .loc 1 6 0
34 0002 054A              ldr r2, .L4
35 0004 1368              ldr r3, [r2]
36 0006 0133              adds    r3, r3, #1
37 0008 1360              str r3, [r2]
38                 .L2:
 5:ej1.c          ****     while (p--) {
39                        .loc 1 5 0
40 000a 044A              ldr r2, .L4+4
41 000c 1368              ldr r3, [r2]
42 000e 591E              subs    r1, r3, #1
43 0010 1160              str r1, [r2]
44 0012 002B              cmp r3, #0
45 0014 F5D1              bne .L3
 7:ej1.c          ****     }
 8:ej1.c          **** }...
46                        .loc 1 8 0
47 0016 7047              bx  lr
```



Introducción

El **ARM** Cortex-M4 es una **CPU**.

El **Kinetis** MK64FN1M0VLL12 es un microcontrolador (**MCU**).





Introducción

¿Qué es ARM?

- **ARM** (Advanced RISC Machine) es una arquitectura de CPU.
- **Arm Holdings** es una empresa que licencia esa arquitectura a otras empresas. Eg: NXP, ST, Microchip, Apple, etc...

¿Qué es una arquitectura?

- La arquitectura de un microprocesador define el “contrato” entre el hardware y el software de una CPU.
- Describe la funcionalidad que el hardware le debe proveer al software, pero no cómo la implementa.

Arquitectura

Table 1. Key elements defined by the Arm architecture	
Feature	**Description**
Instruction set	• The list of available instructions • The function of each instruction, including any exceptions it can trigger • How an instruction is represented in memory, its encoding
Register set	• Set of available registers • The size of the registers • The function of the registers • The reset behaviour of those registers
Exception model	• The different levels of privilege and functions available at each level of privilege • The types of exceptions and what triggers them • What happens on taking or returning from an exception
Memory model	• How memory accesses are ordered • How the caches behave, when and how software must perform explicit maintenance
Debug, trace, and profiling	• How breakpoints are set and triggered • What information can be captured by trace tools and in what format

Arquitectura

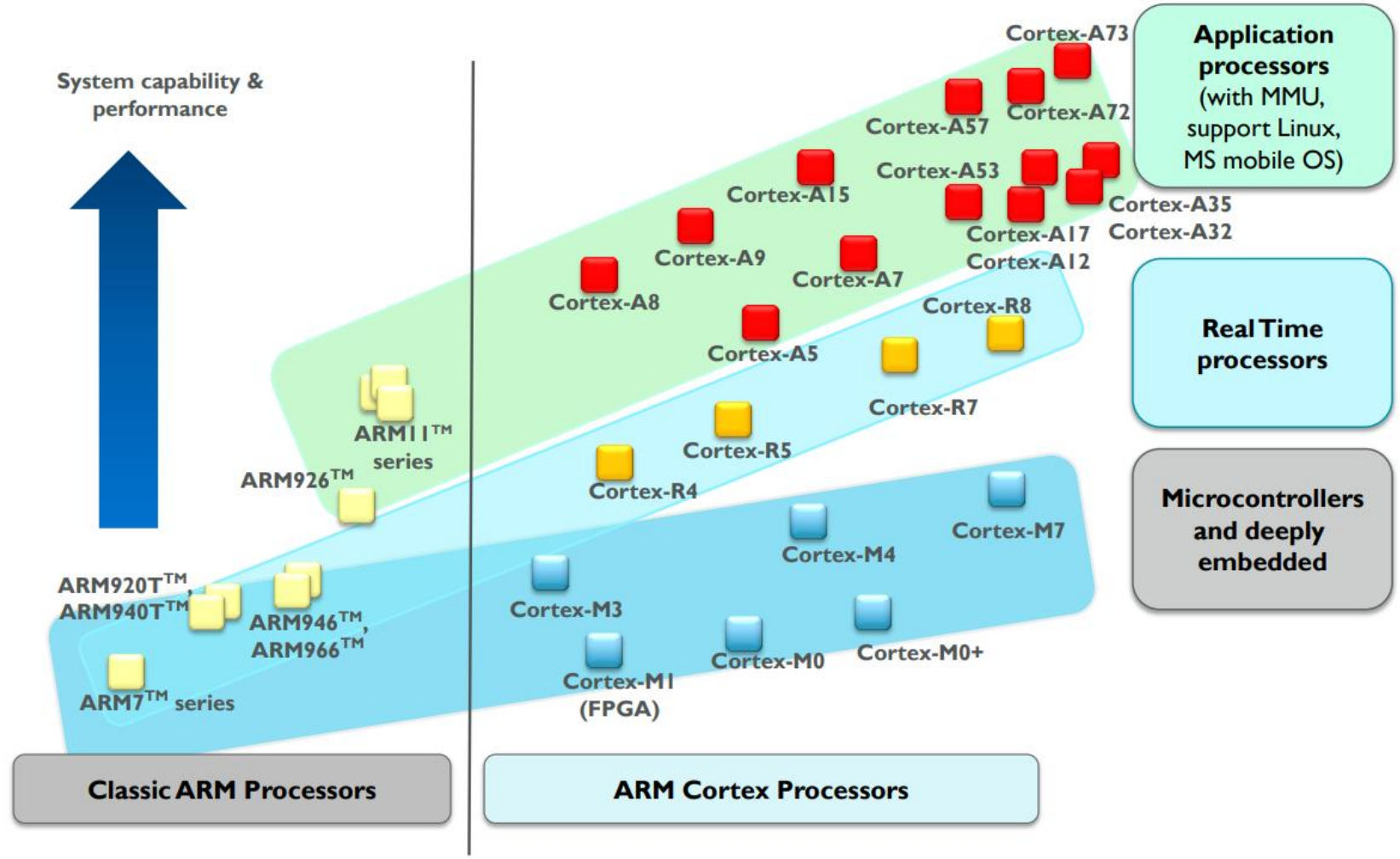


Figure 1: ARM processor family

Arquitectura

	Application processors	Real-time processors	Microcontroller processors
Design	High clock frequency, Long pipeline, High performance, Multimedia support (NEON instruction set extension)	High clock frequency, Long to medium pipeline length, Deterministic (low interrupt latency)	Short pipeline, ultra low power, Deterministic (low interrupt latency)
System features	Memory Management Unit (MMU), cache memory, ARM TrustZone® security extension	Memory Protection Unit (MPU), cache memory, Tightly Coupled Memory (TCM)	Memory Protection Unit (MPU), Nested Vectored Interrupt Controller (NVIC), Wakeup Interrupt Controller (WIC)
Targeted markets	Mobile computing, smart phones, energy-efficient servers, high-end microprocessors	Industrial microcontrollers, automotives, Hard disk controllers, Baseband modem	Microcontrollers, Deeply embedded systems (e.g. sensors, MEMS, mixed signal IC), Internet of Things (IoT)

Table 1: Summary of processor characteristics

Arquitectura

	Descriptions
Cortex-M0	A very small processor (starting from 12K gates) for low cost, ultra low power microcontrollers and deeply embedded applications
Cortex-M0+	The most energy-efficient processor for small embedded system. Similar size and programmer's model to the Cortex-M0 processor, but with additional features like single cycle I/O interface and vector table relocations
Cortex-M1	A small processor design optimized for FPGA designs and provides Tightly Coupled Memory (TCM) implementation using memory blocks on the FPGAs. Same instruction set as the Cortex-M0
Cortex-M3	A small but powerful embedded processor for low-power microcontrollers that has a rich instruction set to enable it to handle complex tasks quicker. It has a hardware divider and Multiply-Accumulate (MAC) instructions. In addition, it also has comprehensive debug and trace features to enable software developers to develop their applications quicker
Cortex-M4	It provides all the features on the Cortex-M3, with additional instructions target at Digital Signal Processing (DSP) tasks, such as Single Instruction Multiple Data (SIMD) and faster single cycle MAC operations. In addition, it also have an optional single precision floating point unit that support IEEE 754 floating point standard
Cortex-M7	High-performance processor for high-end microcontrollers and processing intensive applications. It has all the ISA features available in Cortex-M4, with additional support for double-precision floating point, as well as additional memory features like cache and Tightly Coupled Memory (TCM)

Table 2: The Cortex-M processor family

Instruction Set Architecture

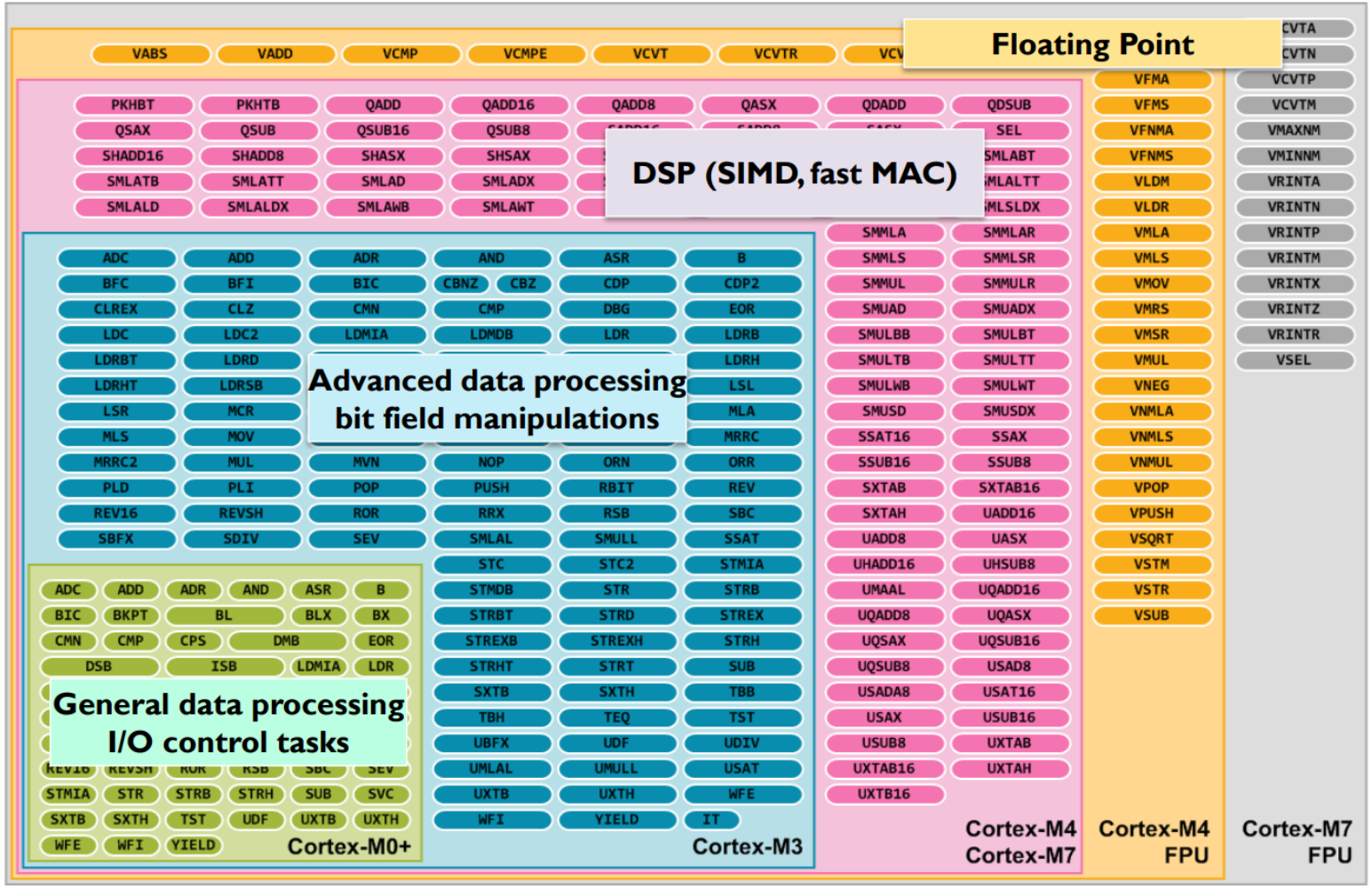
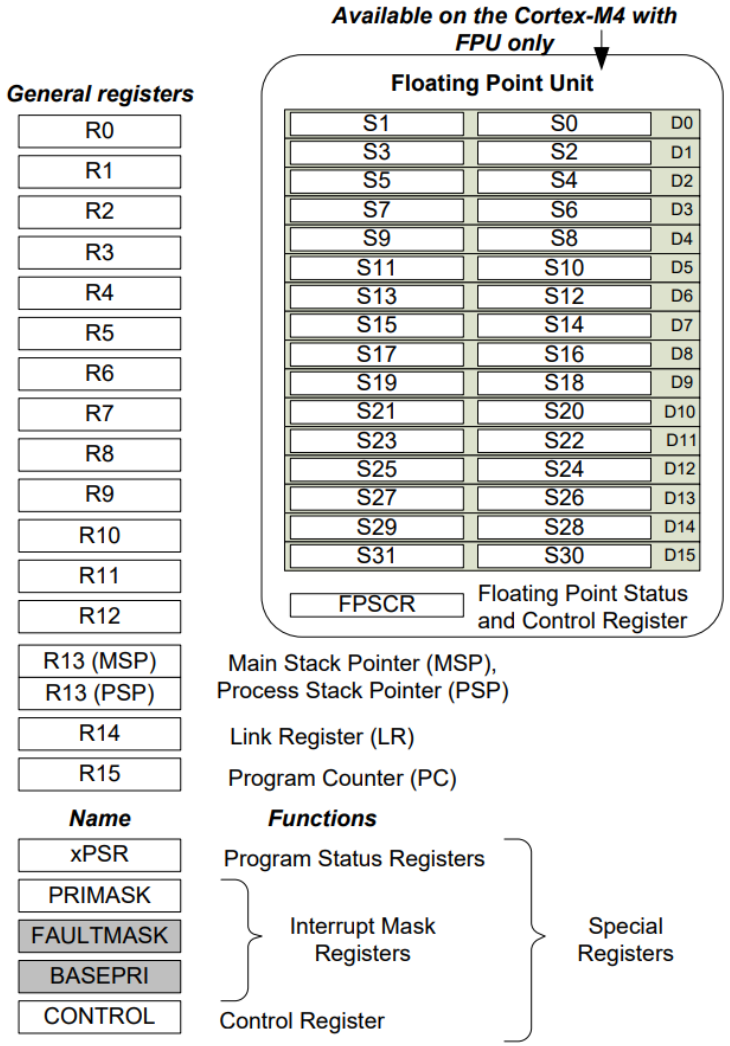


Figure 3: Instruction Set support in the Cortex-M processors

Mapa de Registros

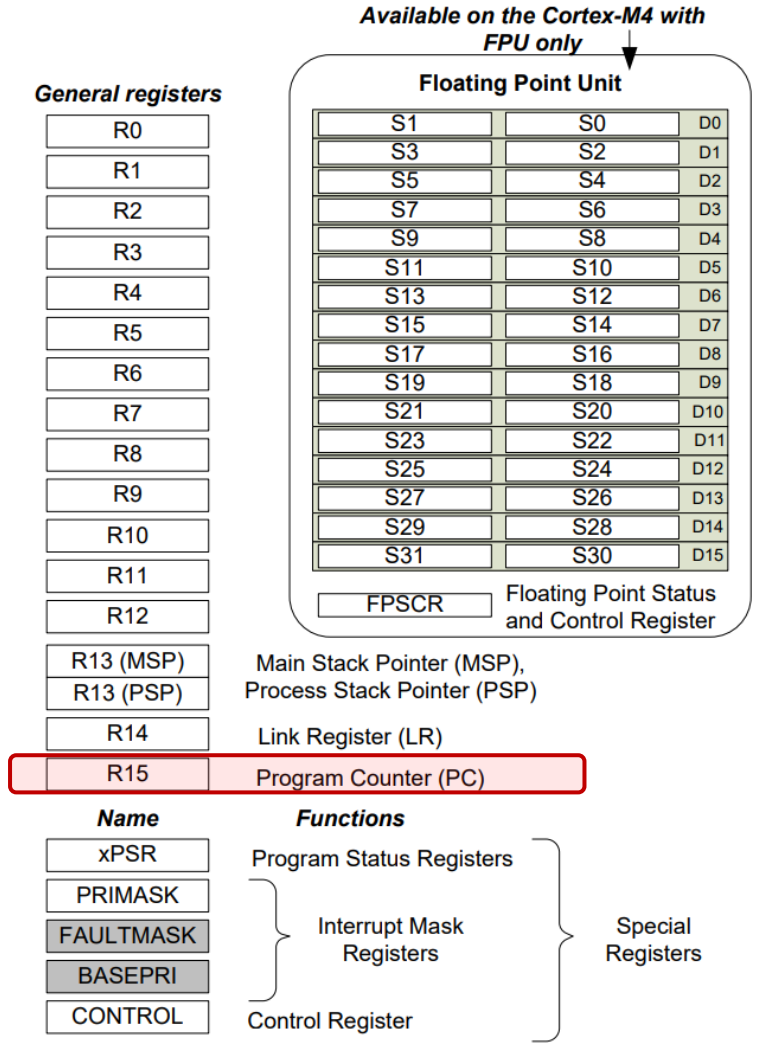
El Cortex-M tiene los siguientes registros de 32 bits:

- 16 registros de propósito general (3 son SP, LR y PC)
- 5 registros especiales
- 32 registros para punto flotante (solo variantes con FPU)



Mapa de Registros

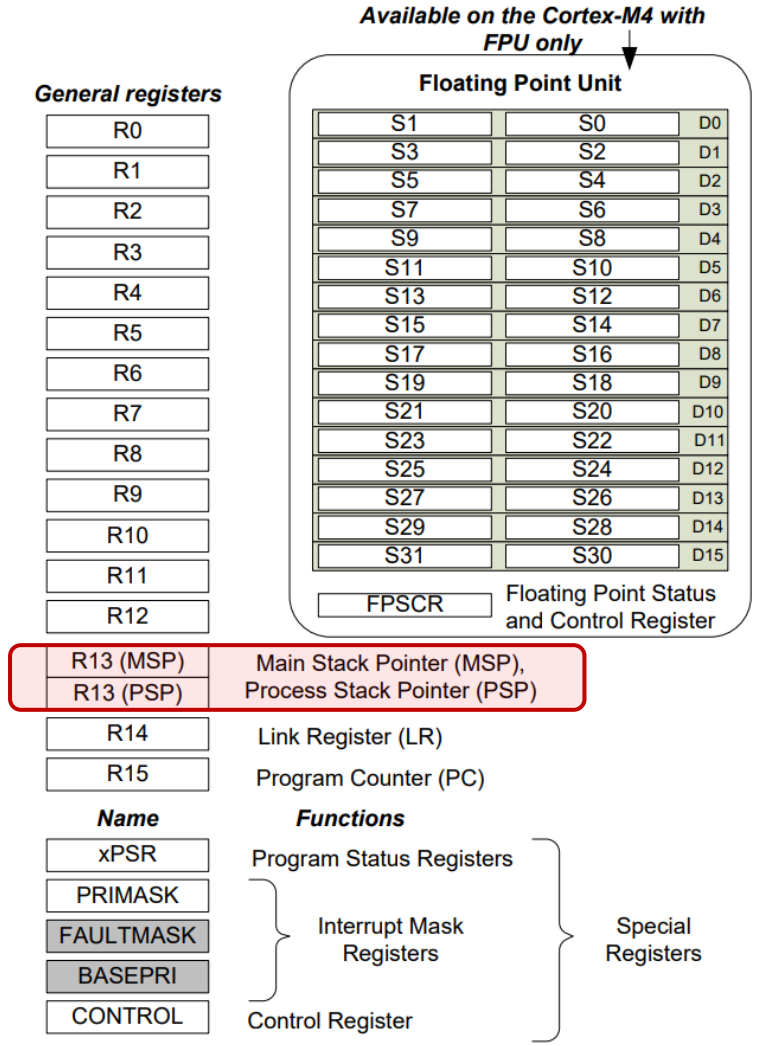
PC → Instrucción que se está ejecutando.



Mapa de Registros

PC → Instrucción que se está ejecutando.

SP → Puntero al stack.



Mapa de Registros: Stack Pointer

Programa principal

```
...

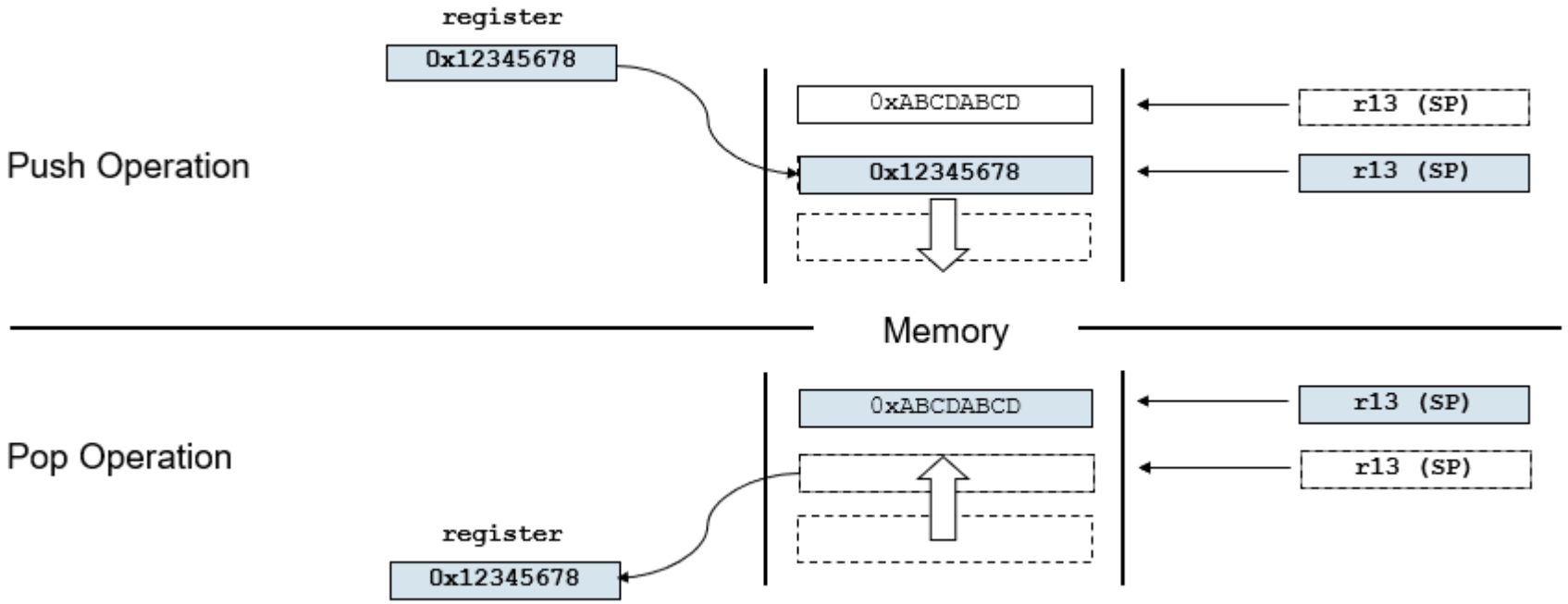
; R4=A, R5=B
BL func1
; Retorno a main
; R4=A, R5=B

...
```

Subrutina

```
func1
    PUSH{R4-R5} ; Guardar R4 y R5 en el stack
    ...         ; Tareas de la subrutina
    POP {R4-R5} ; Restaurar el valor de R4 y R5
    BX LR
```

Mapa de Registros: Stack Pointer

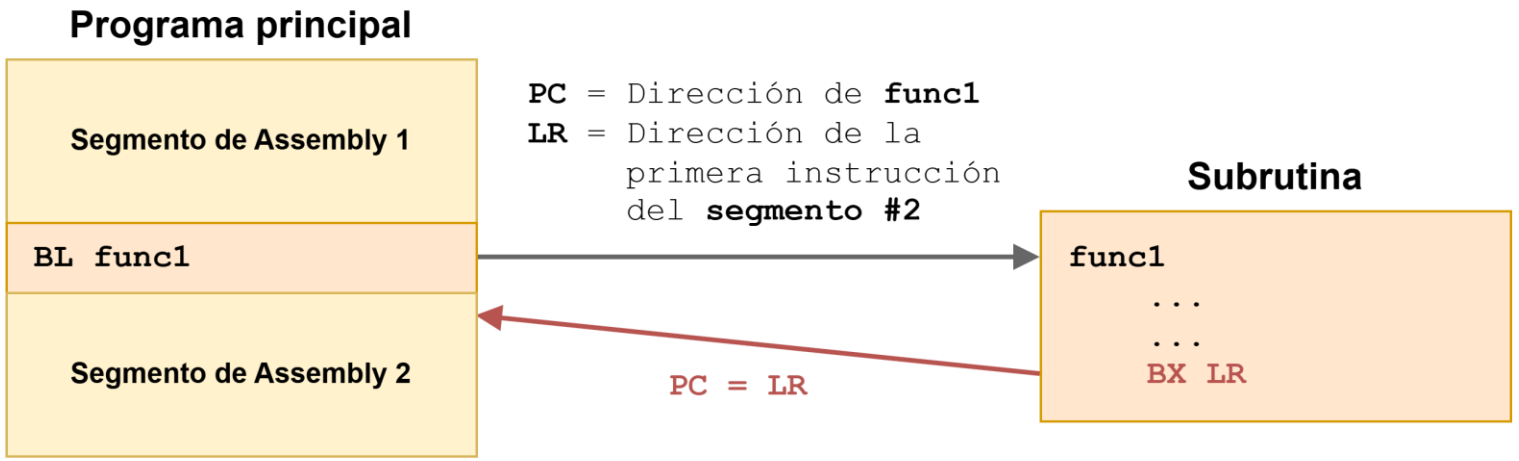


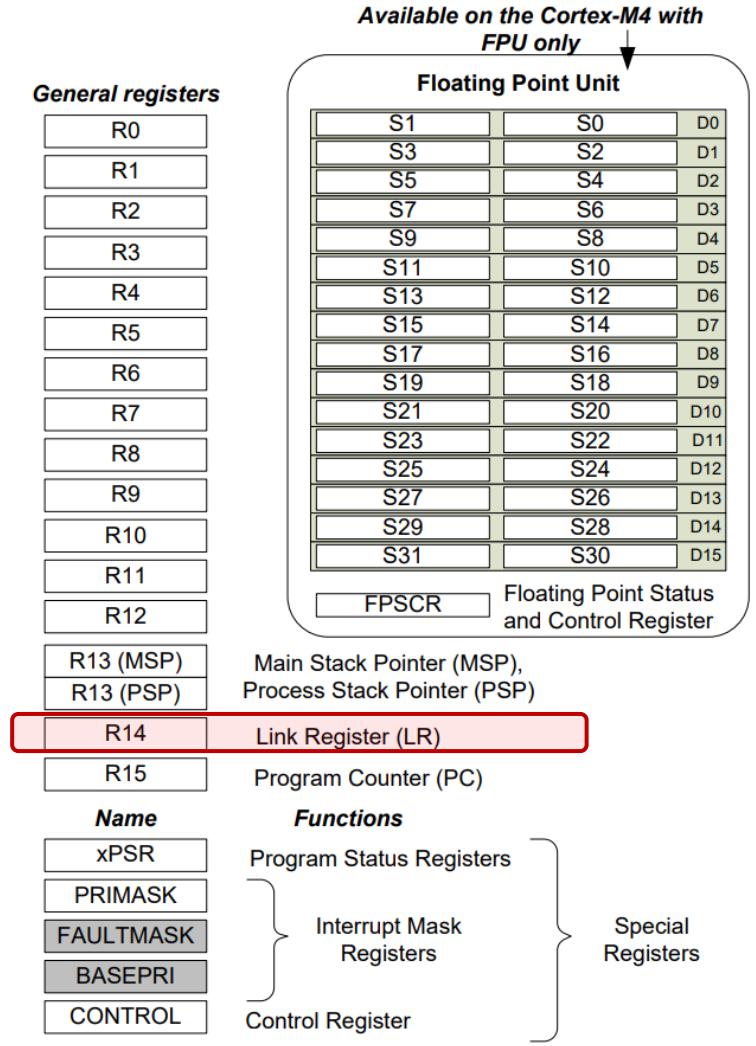
Mapa de Registros

PC → Instrucción que se está ejecutando.

SP → Puntero al stack.

LR → Dirección de retorno de subrutina.





Mapa de Registros

PC → Instrucción que se está ejecutando.

SP → Puntero al stack.

LR → Dirección de retorno de subrutina.

PSR → ALU flags, información de interrupciones y estado de ejecución.

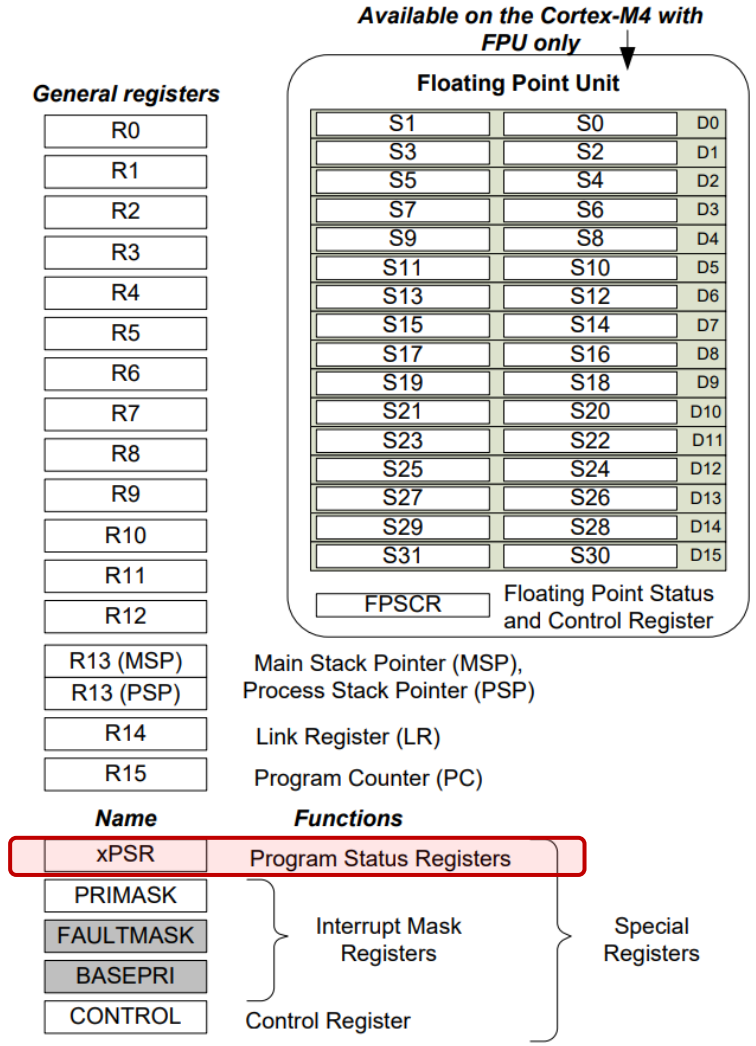
Program Status Register

The *Program Status Register* (PSR) combines:

- *Application Program Status Register* (APSR)
- *Interrupt Program Status Register* (IPSR)
- *Execution Program Status Register* (EPSR).

These registers are mutually exclusive bitfields in the 32-bit PSR. The bit assignments are:

	31	30	29	28	27	26	25	24	23 … 16	15 … 10	9	8	… 0
APSR	N	Z	C	V	Q				Reserved				
IPSR									Reserved				ISR_NUMBER
EPSR	Reserved								ICI/IT	T	Reserved		ICI/IT / Reserved



Mapa de Registros

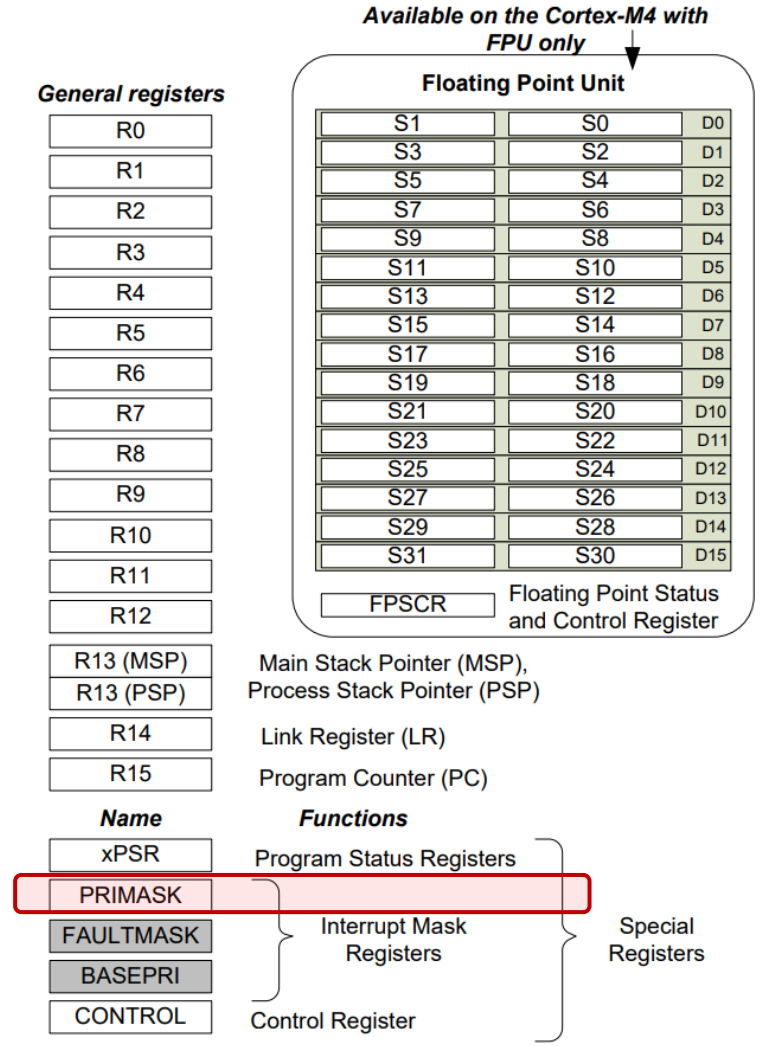
SP → Puntero al stack.

PC → Instrucción que se está ejecutando.

LR → Dirección de retorno de subrutina.

PSR → ALU flags, información de interrupciones y estado de ejecución.

PRIMASK (1 bit) → Bloquea todas las excepciones (incluye interrupciones). Excepto las NMI y HardFault.



Mapa de Registros

SP → Puntero al stack.

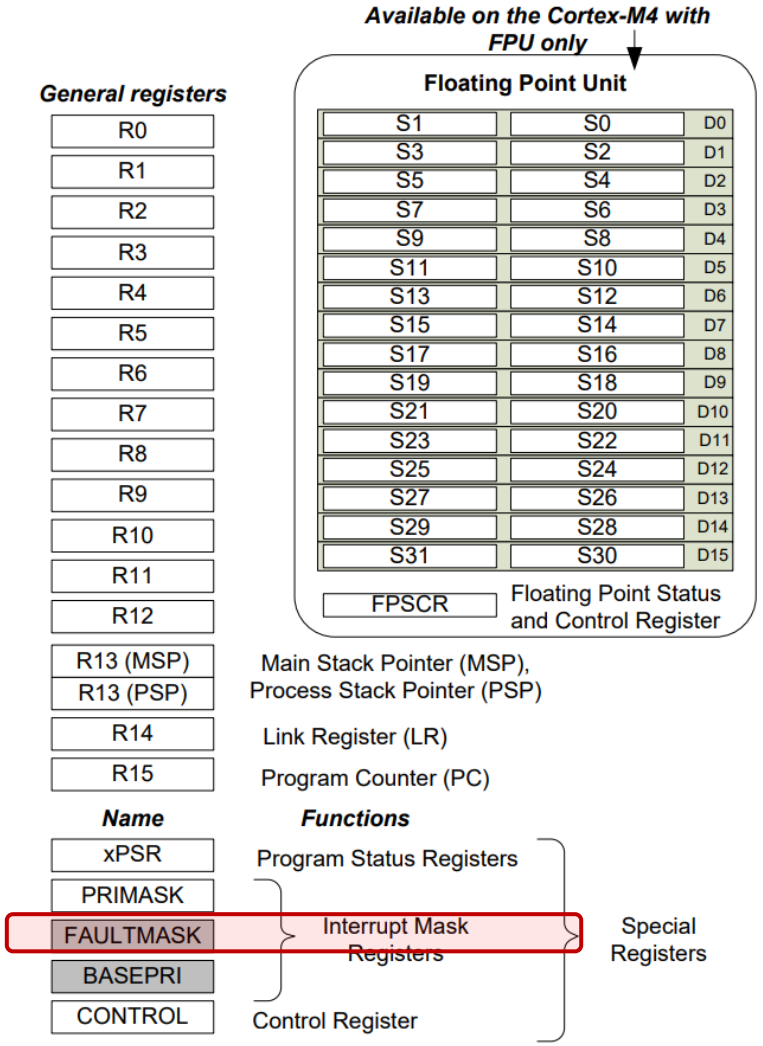
PC → Instrucción que se está ejecutando.

LR → Dirección de retorno de subrutina.

PSR → ALU flags, información de interrupciones y estado de ejecución.

PRIMASK (1 bit) → Bloquea todas las excepciones (incluye interrupciones). Excepto las NMI y HardFault.

FAULTMASK (1 bit) → Bloquea todas las excepciones incluyendo HardFault.



Mapa de Registros

SP → Puntero al stack.

PC → Instrucción que se está ejecutando.

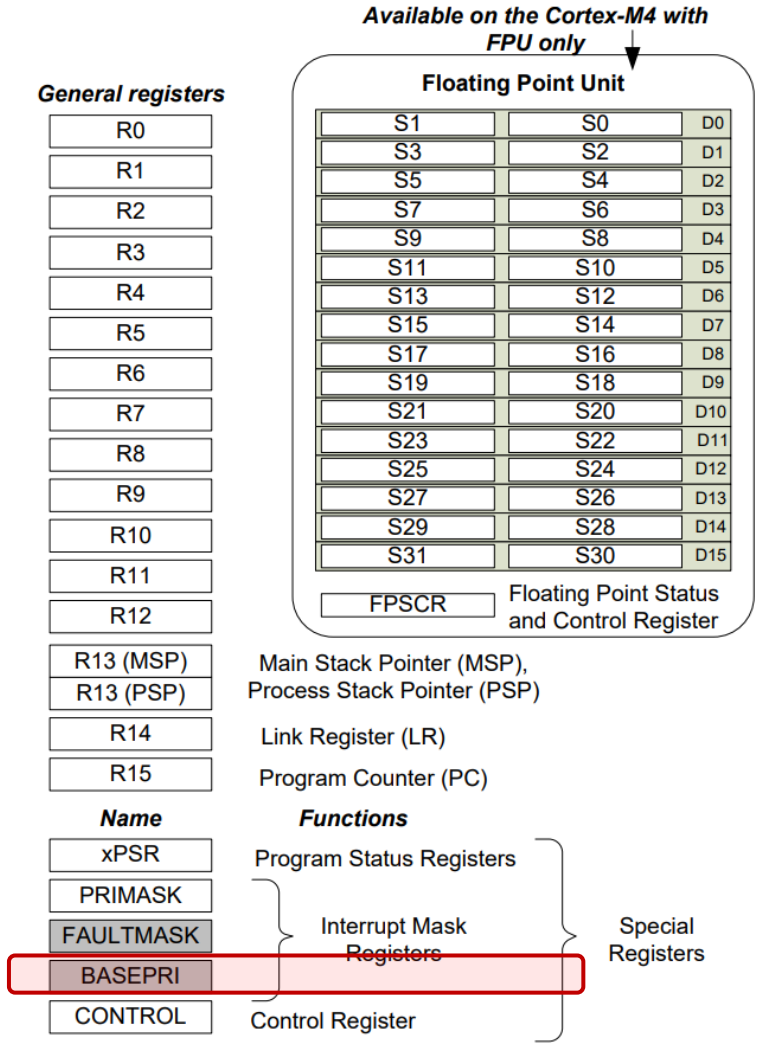
LR → Dirección de retorno de subrutina.

PSR → ALU flags, información de interrupciones y estado de ejecución.

PRIMASK (1 bit) → Bloquea todas las excepciones (incluye interrupciones). Excepto las NMI y HardFault.

FAULTMASK (1 bit) → Bloquea todas las excepciones incluyendo HardFault.

BASEPRI (8 bit) → Bloquea las excepciones con prioridad mayor o igual a su valor.



Mapa de Registros

SP → Puntero al stack.

PC → Instrucción que se está ejecutando.

LR → Dirección de retorno de subrutina.

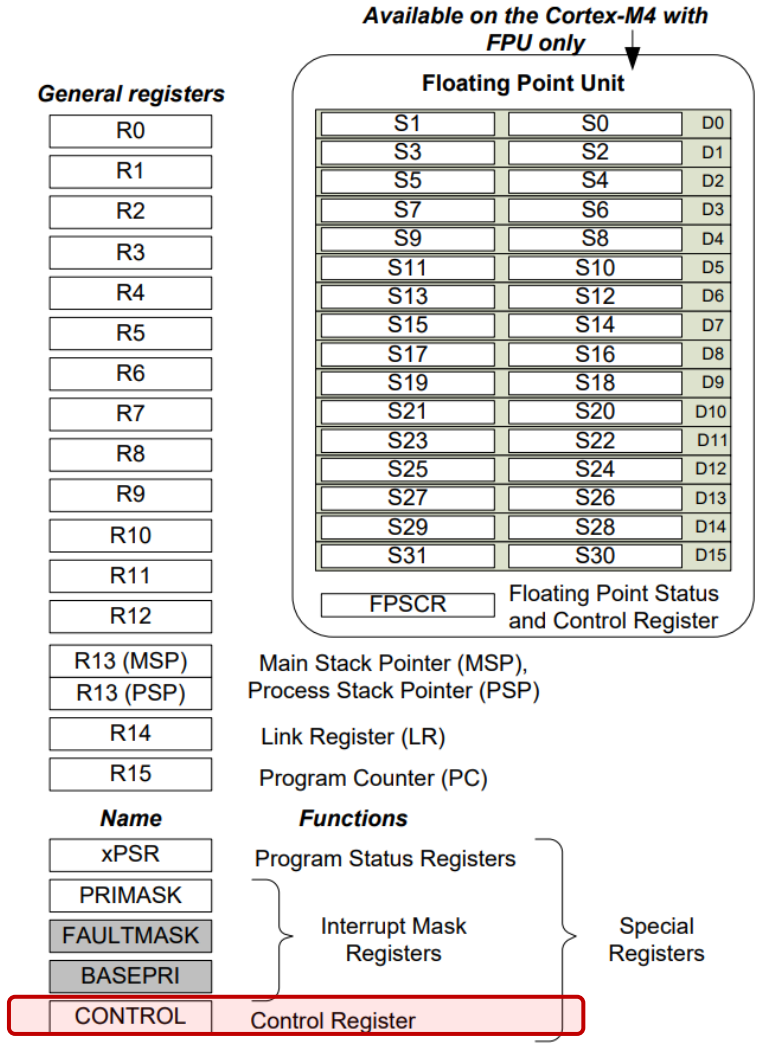
PSR → ALU flags, información de interrupciones y estado de ejecución.

PRIMASK (1 bit) → Bloquea todas las excepciones (incluye interrupciones). Excepto las NMI y HardFault.

FAULTMASK (1 bit) → Bloquea todas las excepciones incluyendo HardFault.

BASEPRI (8 bit) → Bloquea las excepciones con prioridad mayor o igual a su valor.

CONTROL → Selección del SP (MSP/PSP) y otras opciones de control.



Arquitectura: Cortex-M4

- La arquitectura del Cortex-M4 es un “híbrido” entre **Harvard** y **Von Neumann**.
- Tiene buses de memoria separados para código y datos (como la Harvard) pero tiene un único mapa de memoria unificado (como la Von Neumann).
- Notar las características opcionales
- Notar la presencia del NVIC (unificando el controlador de interrupciones en las implementaciones de los distintos fabricantes)
- Notar el Debug Access Port

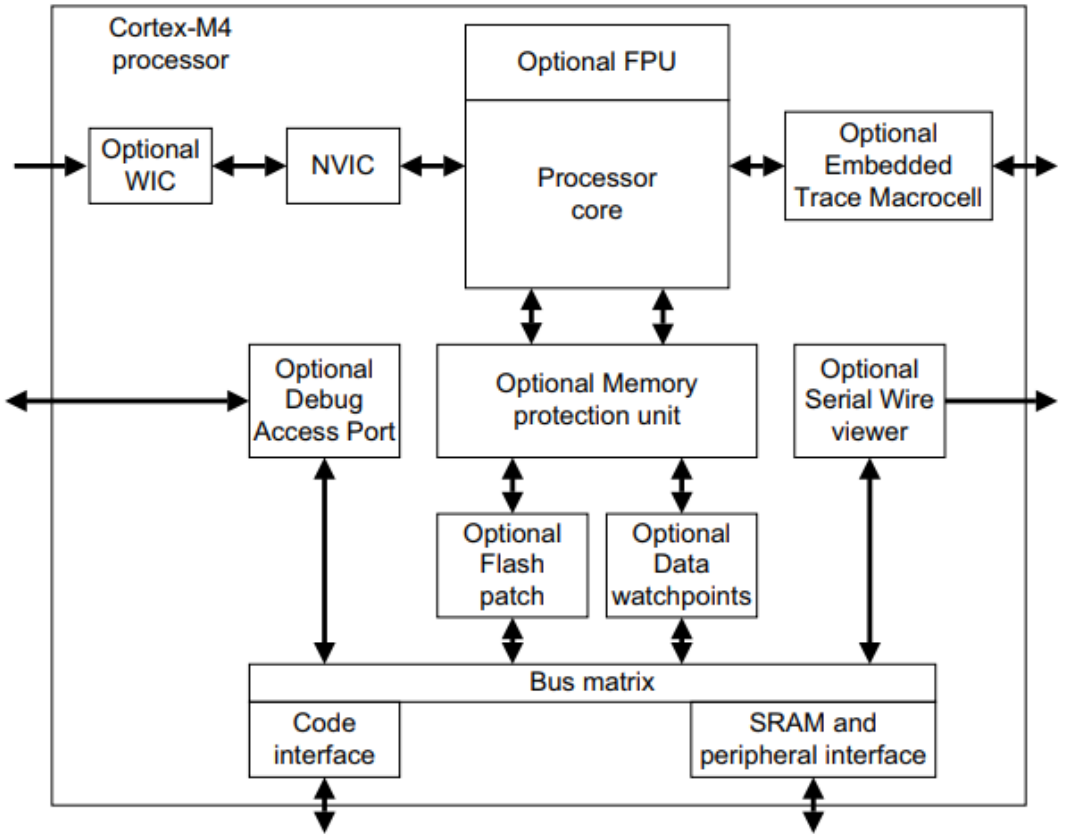


Figure 1-1 Cortex-M4 implementation

Mapa de Memoria

- Todas las implementaciones de ARM Cortex-M4 tienen el mismo mapa de memoria!

¿Qué cosa sí depende del producto/fabricante?

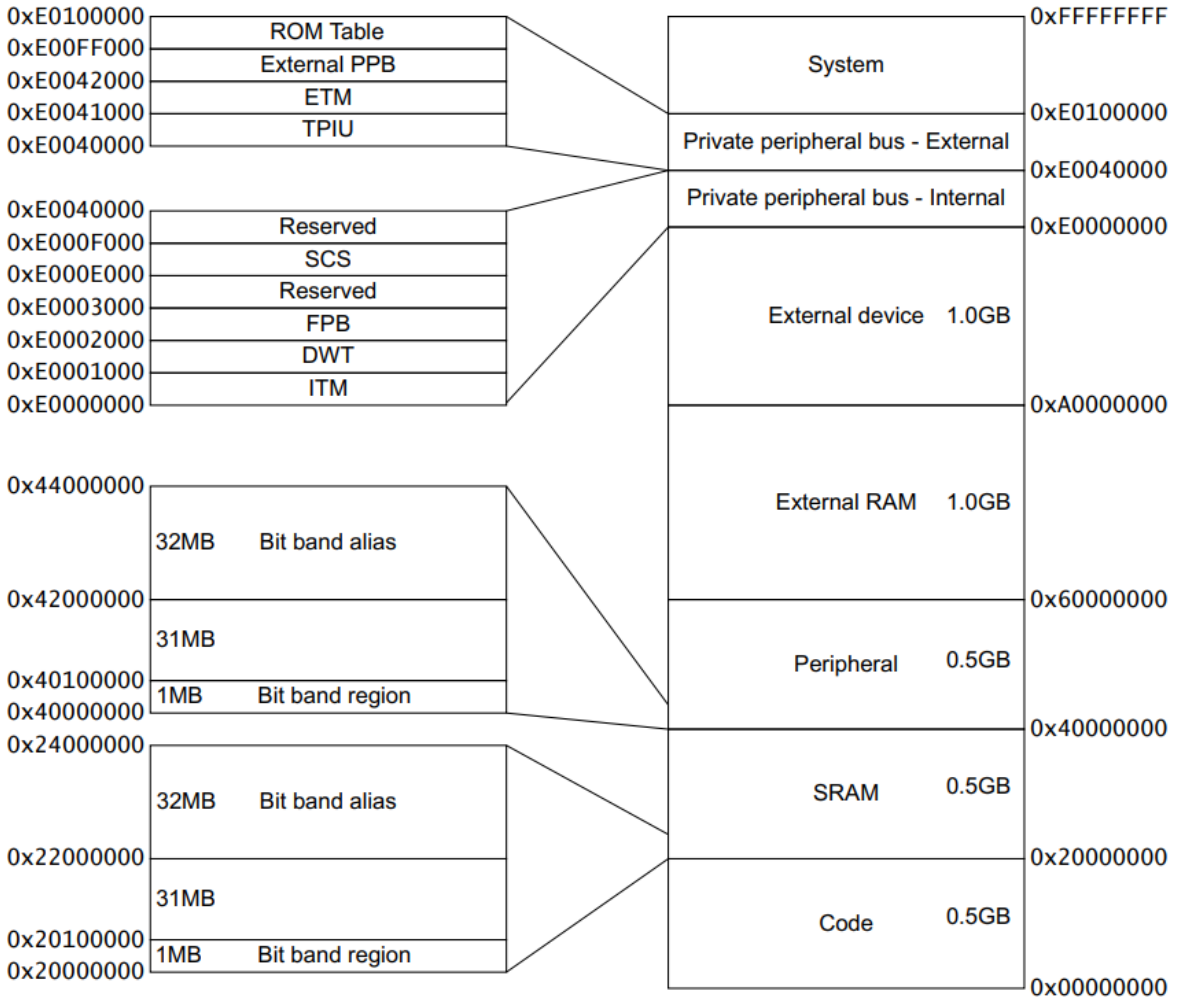


Figure 3-1 System address map

Memoria

¿En qué parte de la memoria se guarda cada variable/función?

const_int

const_string

main()

```c
const int const_int = 42;
const char * const const_string = "Hola";

int main(void) {
    return 0;
}
```

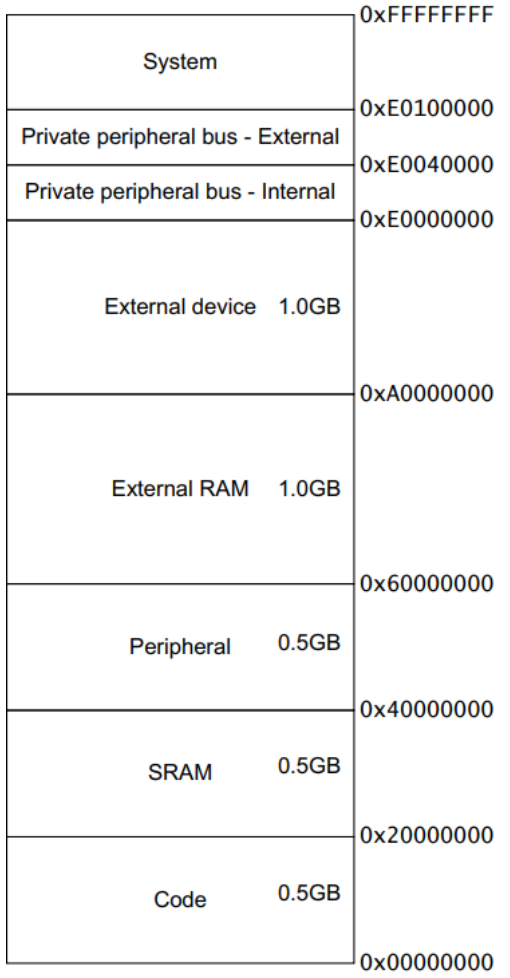


Figure 3-1 System address map

Memoria

¿En qué parte de la memoria se guarda cada variable?

`global_init`

`global_uninit`

```c
int global_uninit;
int global_init = 5;
const int const_int = 42;

int main(void) {
    return 0;
}
```

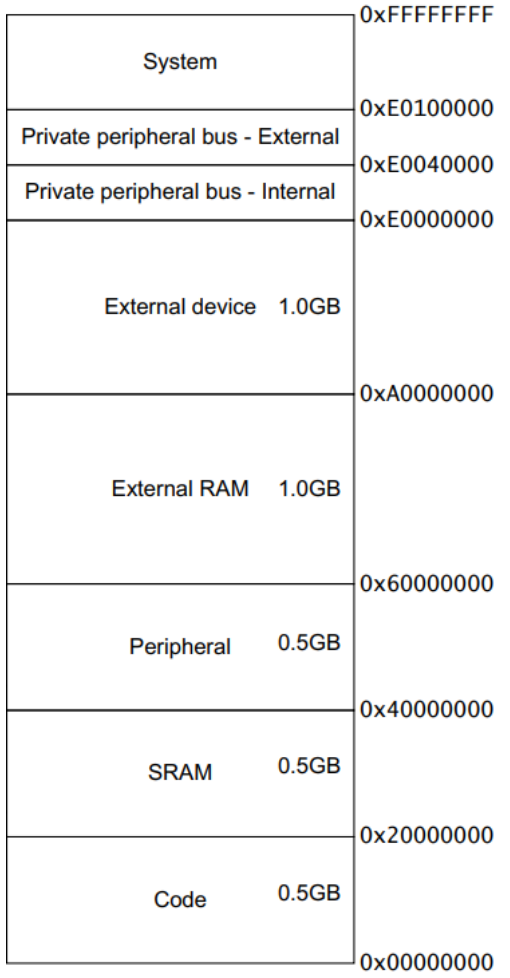


Figure 3-1 System address map

Memoria

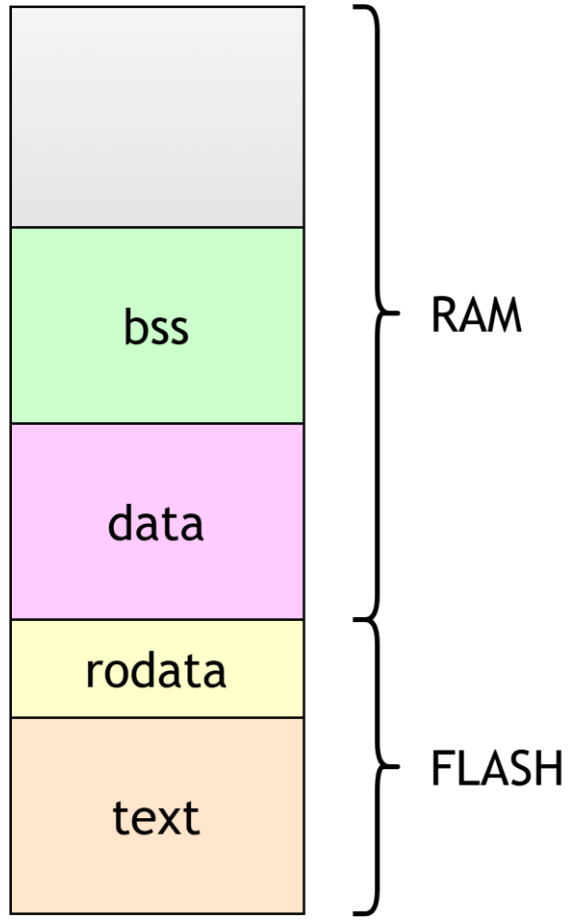
¿En qué parte de la memoria se guarda cada variable?

`global_init`

`global_uninit`

```c
int global_uninit;
int global_init = 5;
const int const_int = 42;

int main(void) {
    return 0;
}
```



Memoria

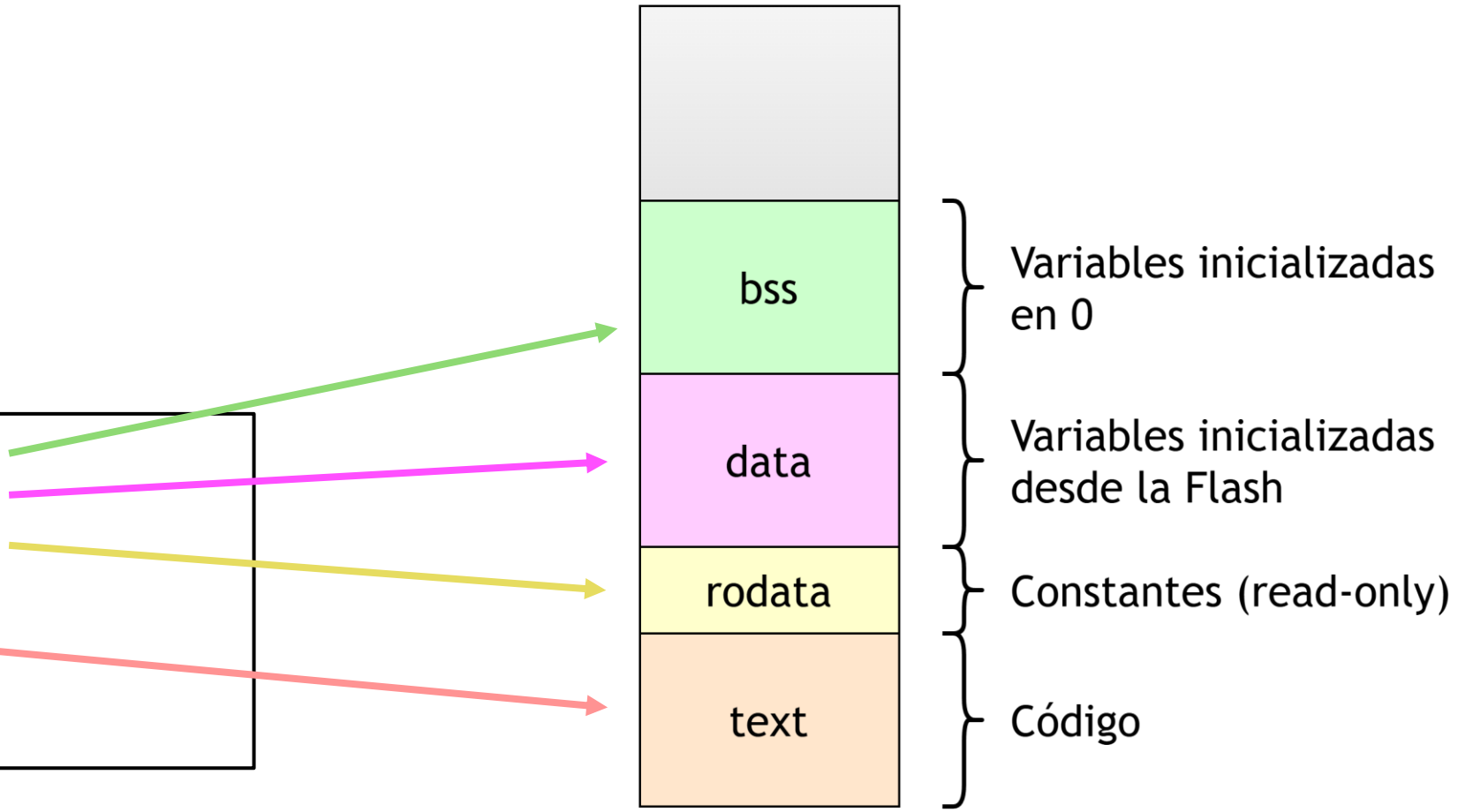
¿En qué parte de la memoria se guarda cada variable?

`global_init`

`global_uninit`

```c
int global_uninit;
int global_init = 5;
const int const_int = 42;

int main(void) {
    return 0;
}
```



Memoria

¿En qué parte de la memoria se guarda cada variable?

```c
#include <stdlib.h>

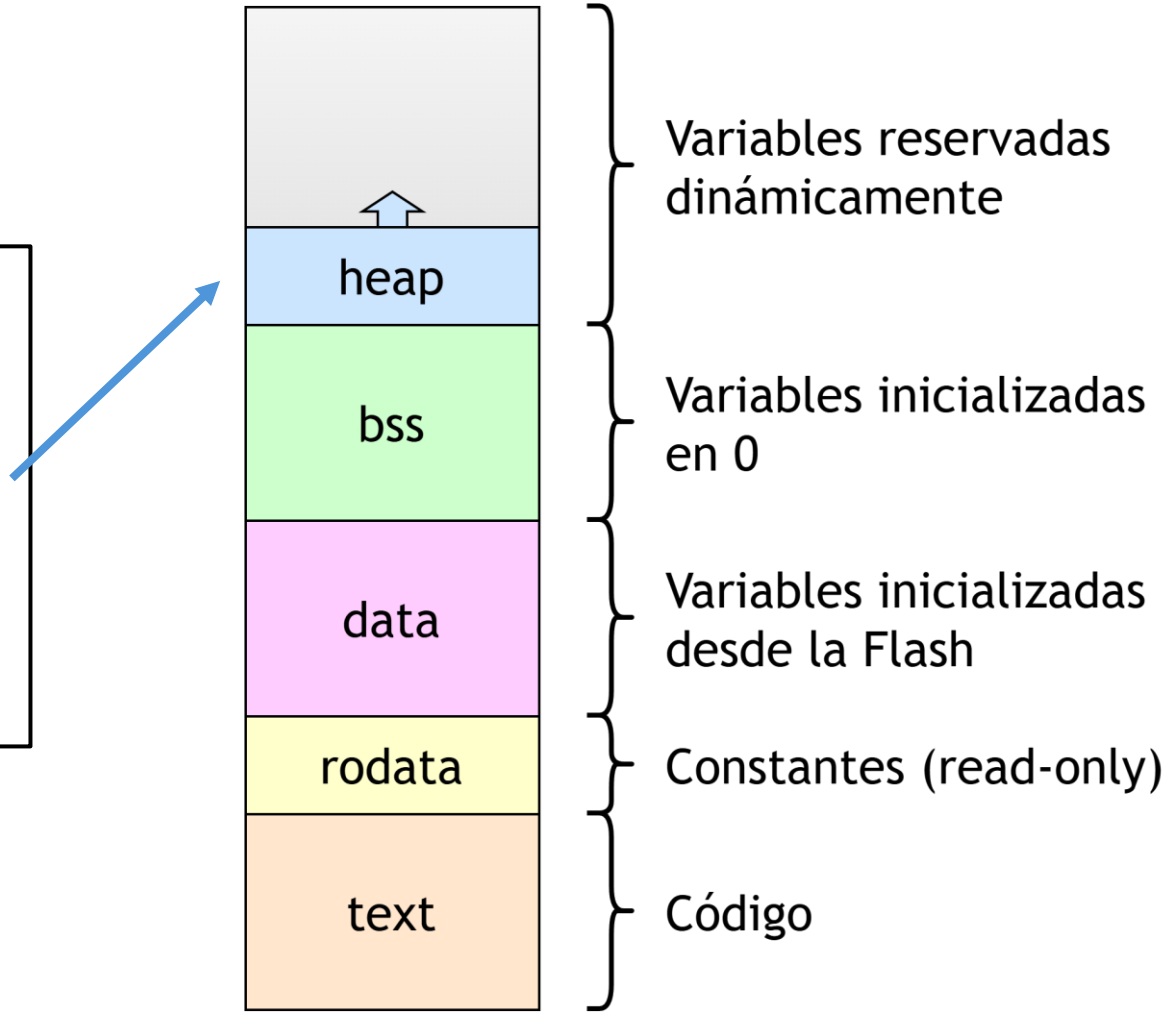

int main(void) {
    int *arr = (int*)malloc(5*sizeof(int));
    arr[0] = 123;
    free(arr);
    return 0;
}
```

Memoria

¿En qué parte de la memoria se guarda cada variable?

```c
#include <stdlib.h>


int main(void) {
    int *arr = (int*)malloc(5*sizeof(int));
    arr[0] = 123;
    free(arr);
    return 0;
}
```



Memoria

¿En qué parte de la memoria se guarda cada variable?

```c
void func1(void) {
    int local1 = 10;
    func2();
}


void func2(void) {
    int local2;
}


int main(void) {
    func2();
    return 0;
}
```

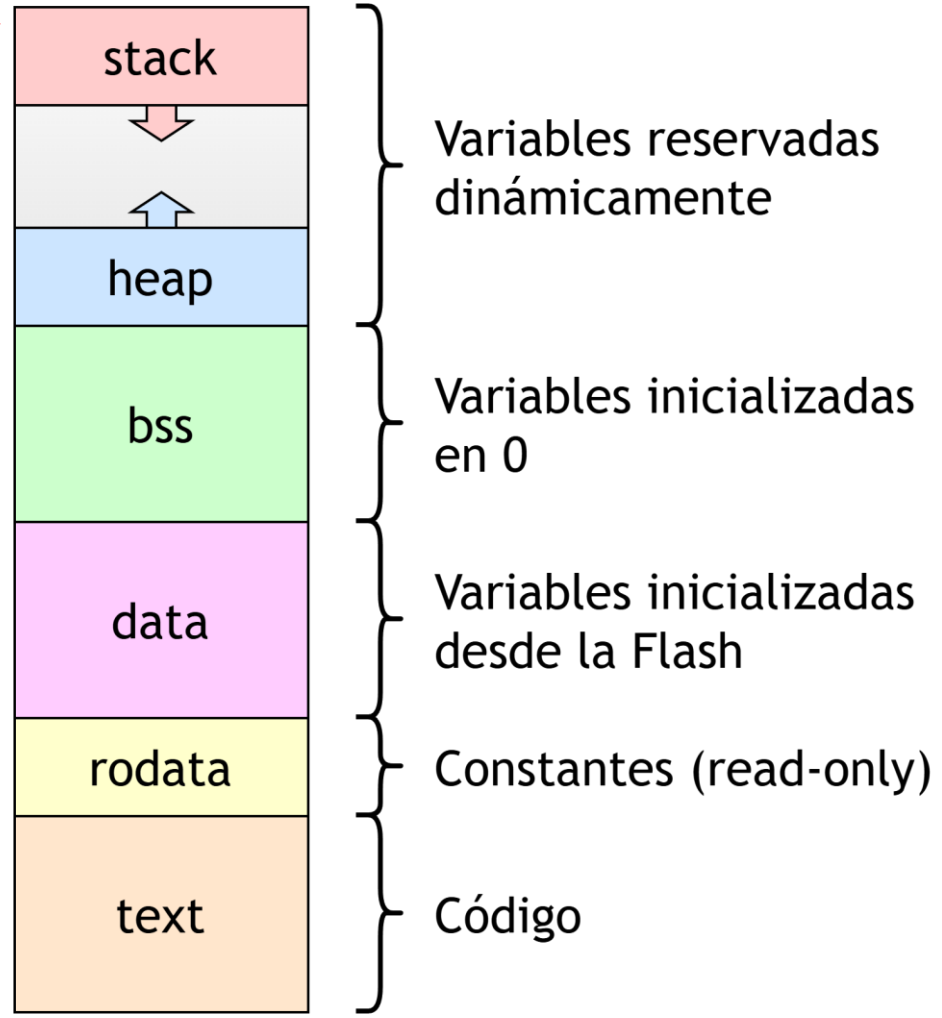
Memoria

¿En qué parte de la memoria se guarda cada variable?

```c
void func1(void) {
    int local1 = 10;
    func2();
}

void func2(void) {
    int local2;
}

int main(void) {
    func1();
    return 0;
}
```



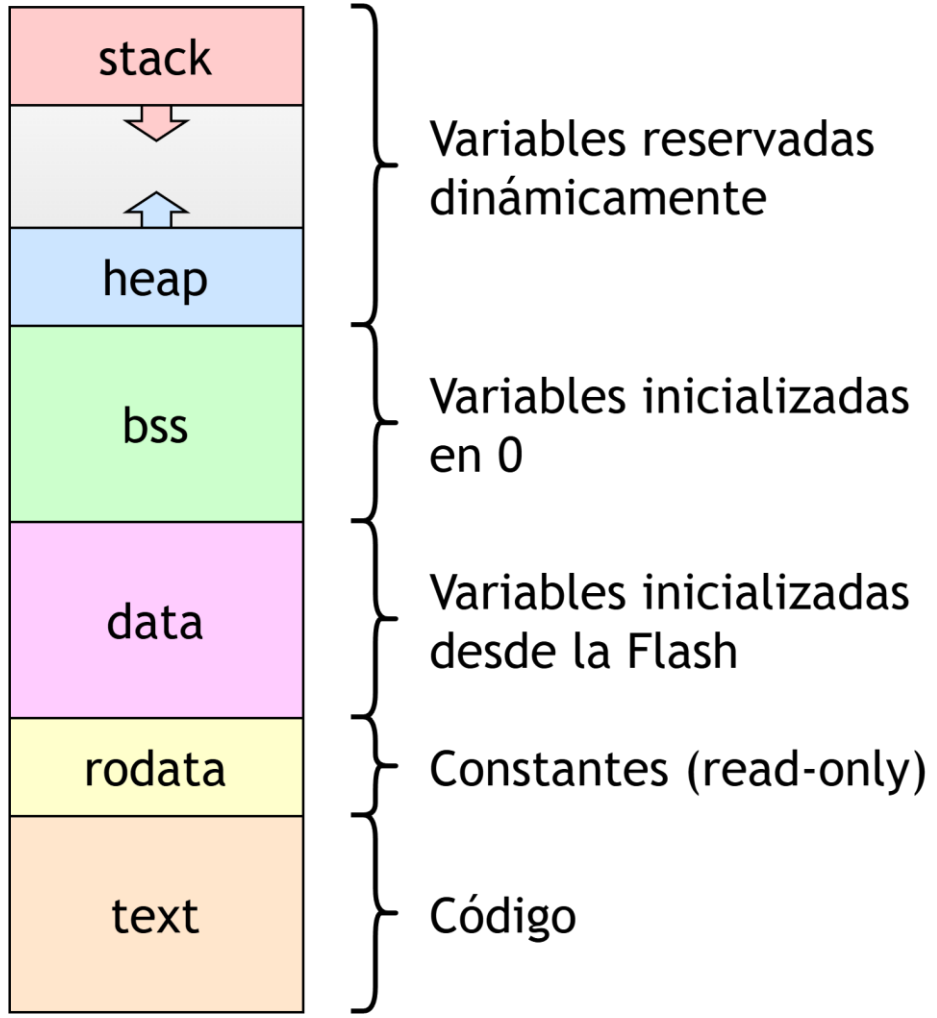
Memoria

¿En qué parte de la memoria se guarda cada variable?

```c
void counter(void) {
    static int static_count = 0;
    static_count++;

    int local_count = 1;
}


int main(void) {
    counter();
    counter();
    return 0;
}
```



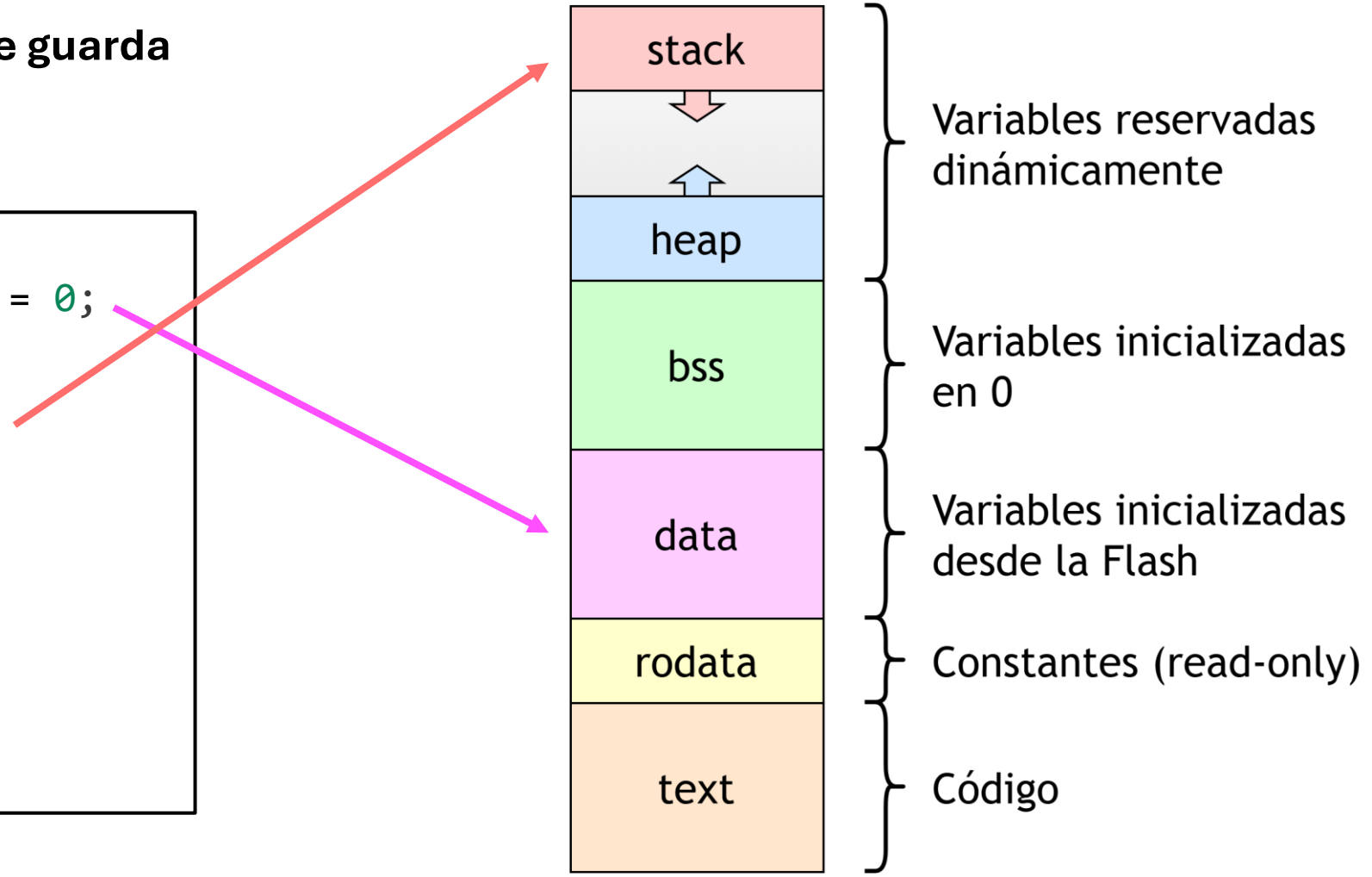
Memoria

¿En qué parte de la memoria se guarda cada variable?

```c
void counter(void) {
    static int static_count = 0;
    static_count++;

    int local_count = 1;
}


int main(void) {
    counter();
    counter();
    return 0;
}
```



Booteo del ARM Cortex-M4

Asumamos que queremos arrancar con un programa súper mínimo:

```
void main(void) {
        while (1) {

        }
}
```

¿Cuál es la secuencia exacta desde que le damos alimentación al micro hasta que ejecuta la primera instrucción del **main**?

¿Hay algo antes del **main**?

Booteo del ARM Cortex-M4

Imaginemos que energizamos el microcontrolador y la tensión comienza a subir.

El reset lo maneja la implementación, no la CPU Cortex-M4.

"***K64*** *Sub-Family Reference Manual*":

6.2.1 Power-on reset (POR)

When power is initially applied to the MCU or when the supply voltage drops below the power-on reset re-arm voltage level (V_{POR}), the POR circuit causes a POR reset condition.

As the supply voltage rises, the LVD circuit holds the MCU in reset until the supply has risen above the LVD low threshold (V_{LVDL}). The POR and LVD bits in SRS0 register are set following a POR.

"***Kinetis*** *K64F Sub-Family Data Sheet*":

2.2.2 LVD and POR operating requirements

Table 2. V_{DD} supply LVD and POR operating requirements

Symbol	Description	Min.	Typ.	Max.	Unit	Notes
V_{POR}	Falling VDD POR detect voltage	0.8	1.1	1.5	V	
V_{LVDL}	Falling low-voltage detect threshold — low range (LVDV=00)	1.54	1.60	1.66	V	

Booteo del ARM Cortex-M4

Después del reset (Power-On Reset u otro), la **CPU**:

1. Carga en el MSP el valor en la posición de memoria 0x00000000
2. Carga en el LR el valor 0xFFFFFFFF
3. Carga en el PC el valor en la posición de memoria 0x00000004

¿Y después?

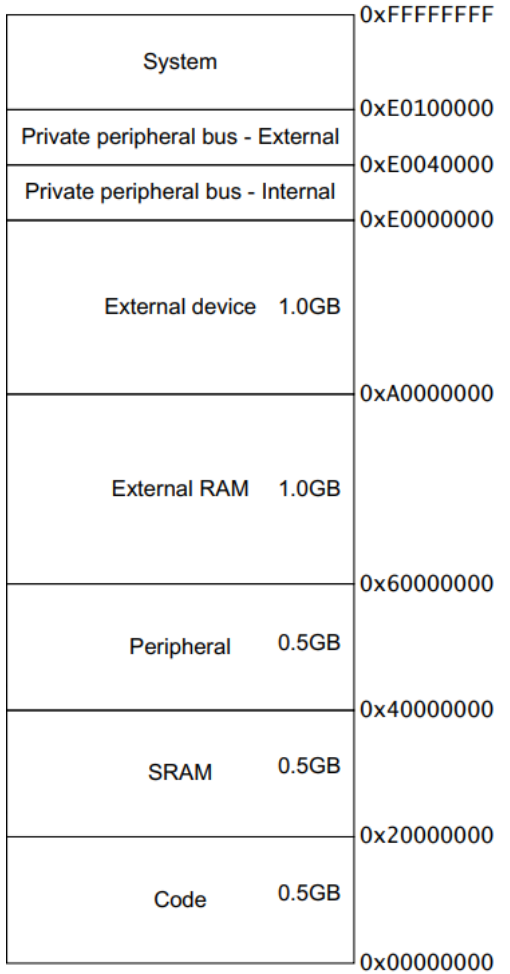


Figure 3-1 System address map

Booteo del ARM Cortex-M4

Después del reset (Power-On Reset u otro), la **CPU**:

1. Carga en el MSP el valor en la posición de memoria 0x00000000
2. Carga en el LR el valor 0xFFFFFFFF
3. Carga en el PC el valor en la posición de memoria 0x00000004

Para bootear un Cortex-M4, tenemos que poner:

- Un valor en la dirección 0 para cargar en el MSP (**¿cuál?**)
- Un valor en la dirección 4 para cargar en PC (**¿cuál?**)
- Los valores iniciales de las variables globales y las constantes.
- ¡Un programa!

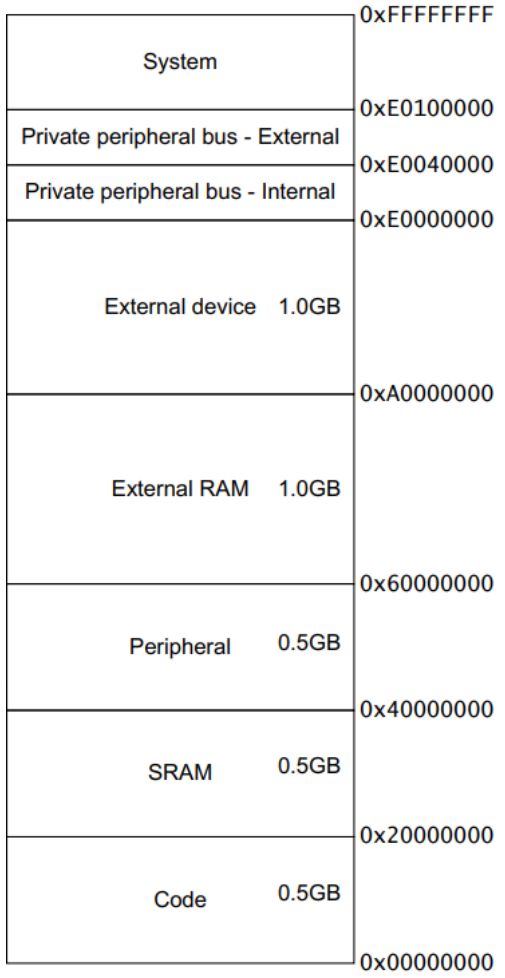


Figure 3-1 System address map

Booteo del ARM Cortex-M4

Después del reset (Power-On Reset u otro), la **CPU**:

1. Carga en el MSP el valor en la posición de memoria 0x00000000
2. Carga en el LR el valor 0xFFFFFFFF
3. Carga en el PC el valor en la posición de memoria 0x00000004

Para bootear un Cortex-M4, tenemos que poner:

- Un valor en la dirección 0 para cargar en el MSP (**¿cuál?**)
- Un valor en la dirección 4 para cargar en PC (**¿cuál?**)
- Los valores iniciales de las variables globales y las constantes.
- ¡Un programa! → **Antes de ejecutar el `main`, el programa debe cargar el valor 0 en las variables no-inicializadas (bss) y copiar los valores de FLASH a RAM para las variables inicializadas (data)**

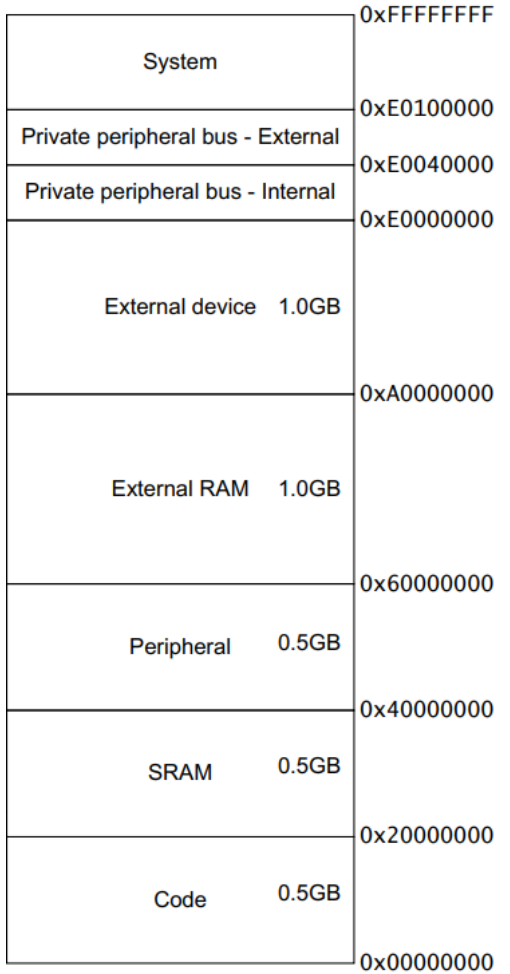


Figure 3-1 System address map

ARM Toolchain

Hagamos un programa mínimo sin IDE ni templates, sólo notepad:

```
void main(void) {
        while (1) {

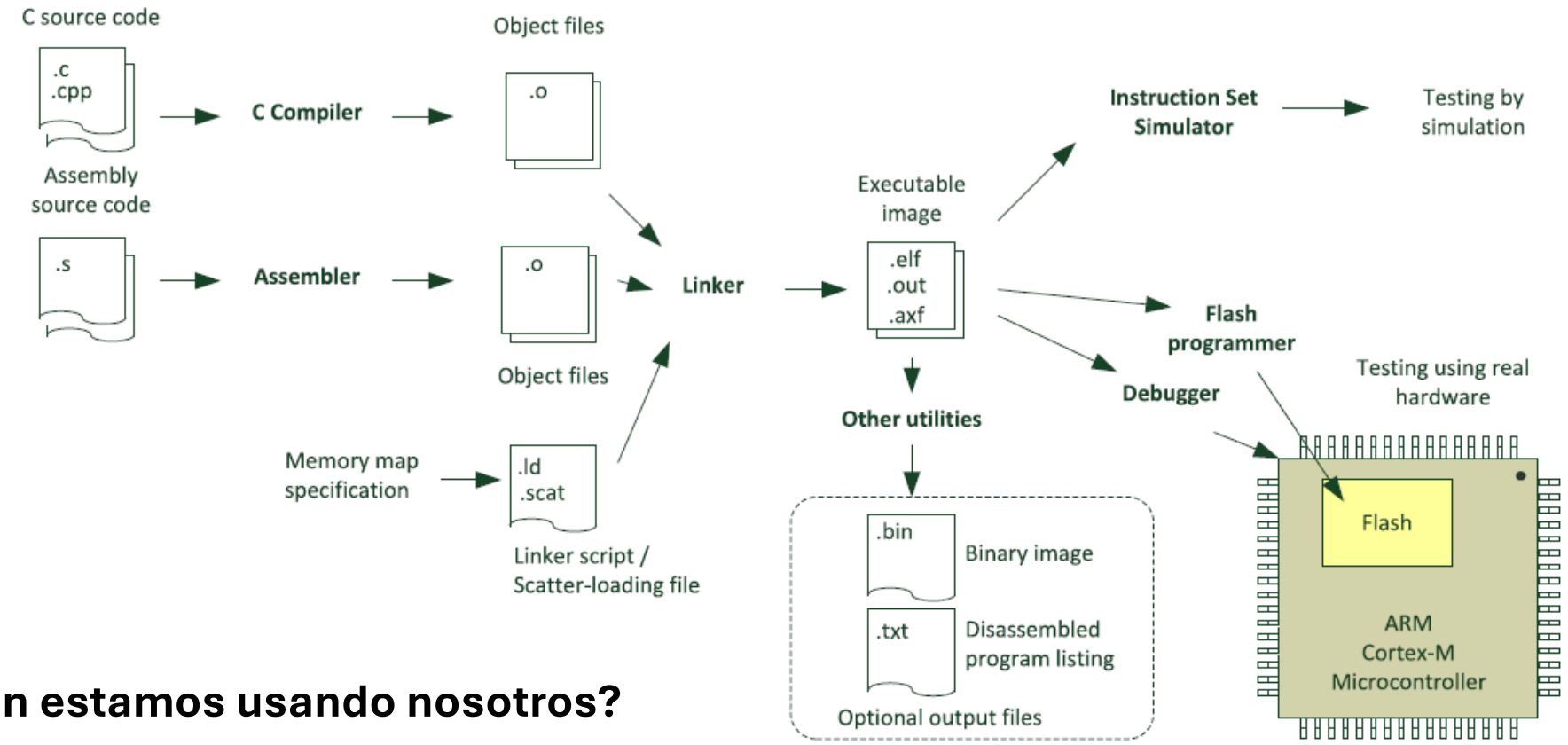
        }
}
```

¿Cómo generamos el contenido binario que debe ir en la FLASH?

Con el Toolchain

ARM Toolchain: Flujo Genérico

"The definitive guide to Cortex-M3/M4"



¿Qué toolchain estamos usando nosotros?

¿Qué hacen el assembler, compiler, linker?

ARM Toolchain: GNU

Compilando con el **gcc**:

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O1 -c ej0.c -o ej0.o

$ arm-none-eabi-objdump -d ej0.o

ej0.o:     file format elf32-littlearm

Disassembly of section .text:

00000000 <main>:
   0:   e7fe            b.n     0 <main>
```

```
void main(void) {
        while (1) {

        }
}
```

¿Por qué la dirección está en 0?

ARM Toolchain: GNU

Linkeando con el **ld**, via el **gcc**:

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles ej0.o -o ej0.elf
/arm-none-eabi/bin/ld: warning: cannot find entry symbol _start;
defaulting to 0000000000008000

$ arm-none-eabi-objdump -d ej0.elf

ej0.elf:     file format elf32-littlearm

Disassembly of section .text:

00008000 <main>:
    8000:       e7fe            b.n     8000 <main>
```

```
void main(void) {
        while (1) {

        }
}
```

¿Está mal que esté en la 8000?

¿Cómo le decimos al toolchain dónde poner el código?

¿A quién se lo tenemos que decir, al compilador o al linker?

ARM Toolchain: Secciones

Compilemos un nuevo programa con gcc, y miremos las **secciones**:

```
$ arm-none-eabi-objdump -td ej1.elf

SYMBOL TABLE:
00008000 l    d  .text   00000000 .text
00008030 l    d  .rodata 00000000 .rodata
00018034 l    d  .data   00000000 .data
00018038 l    d  .bss    00000000 .bss
00018034 l     O .data   00000004 p
00008030 l     O .rodata 00000004 r
0001803c g       .bss    00000000 _bss_end__
00018038 g       .bss    00000000 __bss_start__
0001803c g       .bss    00000000 __bss_end__
00018038 g       .bss    00000000 __bss_start
00008000 g     F .text   00000030 main
0001803c g       .bss    00000000 __end__
00018038 g       .data   00000000 _edata
0001803c g       .bss    00000000 _end
00018038 g     O .bss    00000004 q
00018034 g       .data   00000000 __data_start
```

```
static int p = 100;
int q;
static const int r = 9;

void main(void) {
    while (p-- > r) {
        q++;
    }
}
```

ARM Toolchain: Secciones

El **linker** está encargado de asignar las direcciones finales de los elementos del programa.

El **compilador** asigna **secciones** a pedazos del código, y el **linker** asigna direcciones a las secciones.

```
static int p = 100;
int q;
static const int r = 9;

void main(void) {
    while (p-- > r) {
        q++;
    }
}
```

Secciones “estándar” o default:

.bss: Variables inicializadas en 0

.data: Variables inicializadas != 0 (**RW**)

.rodata: Variables inicializadas != 0 (**RO**)

.text: Código

ARM Toolchain: Secciones

Script de **linker**:

```
MEMORY
{
    flash :       ORIGIN = 0x00000000, LENGTH = 1024k
    ram :         ORIGIN = 0x20000000, LENGTH = 256K
}

SECTIONS
{
    .text :
    {
        . = ALIGN(4);
        *(.text*)
        *(.rodata*)
    } > flash

            … continuado en la siguiente …
```

```
    .data :
    {
        . = ALIGN(4);
        _data_start = .;
        *(.data*)
        . = ALIGN(4);
        _data_end = .;
    } > ram AT > flash

    _data_init_values = LOADADDR(.data);

    .bss (NOLOAD) :
    {
        . = ALIGN(4);
        _bss_start = . ;
        *(.bss*)
        *(COMMON)
        . = ALIGN(4);
        _bss_end = . ;
    } > ram
}
```

ARM Toolchain: Secciones

Compilo y linkeo con **gcc**, usando el script:

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles -T ej1.ld ej1.c -o
ej1.elf

$ arm-none-eabi-objdump -td ej1.elf

SYMBOL TABLE:
00000000 l    d  .text  00000000 .text
20000000 l    d  .data  00000000 .data
20000004 l    d  .bss   00000000 .bss
20000000 l     O .data  00000004 p
00000030 l     O .text  00000004 r
20000004 g       .bss   00000000 _bss_start
20000008 g       .bss   00000000 _bss_end
20000000 g       .data  00000000 _data_start
00000034 g       *ABS*  00000000 _data_init_values
20000004 g       .data  00000000 _data_end
00000000 g     F .text  00000030 main
20000004 g     O .bss   00000004 q
```

```
static int p = 100;
int q;
static const int r = 9;

void main(void) {
    while (p-- > r) {
        q++;
    }
}
```

El linker **provee** los símbolos

_bss_start, _bss_end, _data_start, _data_end

pero **nadie** se encarga de usarlos...

ARM Toolchain: Secciones

Extraigo el **binario** que va en flash:

```
$ arm-none-eabi-objcopy -O binary ej1.elf ej1.bin

Offset      0  1  2  3  4  5  6  7   8  9  A  B  C  D  E  F

00000000   80 B4 00 AF 04 E0 08 4B  1B 68 01 33 06 4A 13 60   €´ ¯ à K h 3 J `
00000010   06 4B 1B 68 5A 1E 05 49  0A 60 09 22 93 42 F2 DC    K hZ  I ` "“BòÜ
00000020   00 BF BD 46 80 BC 70 47  04 00 00 20 00 00 00 20    ¿½F€¼pG
00000030   09 00 00 00 64 00 00 00                             d
```

```
SYMBOL TABLE:
00000000 l    d  .text  00000000 .text
20000000 l    d  .data  00000000 .data
20000004 l    d  .bss   00000000 .bss
20000000 l     O .data  00000004 p
00000030 l     O .text  00000004 r
20000004 g       .bss   00000000 _bss_start
20000008 g       .bss   00000000 _bss_end
20000000 g       .data  00000000 _data_start
00000034 g       *ABS*  00000000 _data_init_values
20000004 g       .data  00000000 _data_end
00000000 g     F .text  00000030 main
20000004 g     O .bss   00000004 q
```

```
static int p = 100;
int q;
static const int r = 9;

void main(void) {
    while (p-- > r) {
        q++;
    }
}
```

¿Qué falta configurar?

Recordar el procedimiento de booteo del Cortex-M4

Booteo

Después del reset (Power-On Reset u otro), la **CPU**:

1. Carga en el MSP el valor en la posición de memoria 0x00000000
2. Carga en el LR el valor 0xFFFFFFFF
3. Carga en el PC el valor en la posición de memoria 0x00000004

Para bootear un Cortex-M4, tenemos que poner:

- Un valor en la dirección 0 para cargar en el MSP (**¿cuál?**)
- Un valor en la dirección 4 para cargar en PC (**¿cuál?**)
- Los valores iniciales de las variables globales y las constantes.
- ¡Un programa! → **Antes de ejecutar el `main`, el programa debe cargar el valor 0 en las variables no-inicializadas (bss) y copiar los valores de FLASH a RAM para las variables inicializadas (data)**

Booteo

Inicialización pre-main:

init.c

```
#include <string.h>

void init(void) {
    extern char _bss_start[];
    extern char _bss_end[];

    extern char _data_start[];
    extern char _data_end[];
    extern char _data_init_values[];

    memset(_bss_start, 0, (_bss_end - _bss_start));

    memcpy(_data_start, _data_init_values, (_data_end - _data_start));
}
```

Booteo

Escribimos los valores correctos en las direcciones 0 y 4:

startup.S

```
    .cpu cortex-m4
    .global _reset_vector

/* sección “.vectors”, allocatable */
.section .vectors, "a"
    .word _sp_init_value
    .word _reset_vector

.section .text
_reset_vector:
    /* esto podría estar en ASM */
    bl init

    bl main

    b .
```

ej2.ld

```
MEMORY
{
    flash : org = 0, len = 1024k
    ram : org = 0x20000000, len = 256K
}

ENTRY(_reset_vector)

SECTIONS
{
    .vectors :
    {
        *(.vectors*)
    } > flash

… etc …

    _sp_init_value = org(ram) + len(ram);
}
```

Booteo

Compilamos cada archivo, **linkeamos** todos juntos, y observamos la salida:

```
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c startup.S -o startup.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c init.c -o init.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -O0 -c ej2.c -o ej2.o
$ arm-none-eabi-gcc -mcpu=cortex-m4 -nostartfiles -T ej2.ld startup.o init.o ej2.o -o ej2.elf
$ arm-none-eabi-objdump -td ej2.elf > ej2.dump.txt
$ arm-none-eabi-objcopy -O binary ej2.elf ej2.bin
```


ej2.dump.txt

```
Offset      0  1  2  3  4  5  6  7   8  9  A  B  C  D  E  F

00000000   00 00 04 20 08 00 00 00  00 F0 04 F8 00 F0 20 F8           ð ø ð ø
00000010   FE E7 00 00 80 B5 00 AF  08 4A 09 4B D3 1A 1A 46   þç  €µ ¯ J KÓ  F
00000020   00 21 07 48 00 F0 C6 F8  06 4A 07 4B D3 1A 1A 46    ! H ðÆø J KÓ  F
00000030   06 49 05 48 00 F0 24 F8  00 BF 80 BD 08 00 00 20    I H ð$ø ¿€½

                  ..........................................................................

00000200   10 32 B2 42 F8 D1 04 F0  0F 04 01 37 03 2C 03 EB    2²BøÑ ð   7 , ë
00000210   07 13 0D D9 1E 46 22 46  04 3A 03 2A 46 F8 04 5B      Ù F"F : *Fø [
00000220   FA D8 22 1F 22 F0 03 02  04 32 13 44 04 F0 03 04   úØ" "ð   2 D ð
00000230   2C B1 C9 B2 1C 44 03 F8  01 1B 9C 42 FB D1 F0 BC   ,±É² D ø  œBûÑð¼
00000240   70 47 14 46 03 46 C5 E7  09 00 00 00 64 00 00 00   pG F FÅç     d
```

Kinetis Startup: startup_mk64f12.c

```
//*****************************************************************************
// The vector table.
// This relies on the linker script to place at correct location in memory.
//*****************************************************************************
extern void (* const g_pfnVectors[])(void);
extern void * __Vectors __attribute__ ((alias ("g_pfnVectors")));

__attribute__ ((used, section(".isr_vector")))
void (* const g_pfnVectors[])(void) = {
    // Core Level - CM4
    &_vStackTop,                        // The initial stack pointer
    ResetISR,                           // The reset handler
    NMI_Handler,                        // The NMI handler
    HardFault_Handler,                  // The hard fault handler
    MemManage_Handler,                  // The MPU fault handler
    BusFault_Handler,                   // The bus fault handler
    UsageFault_Handler,                 // The usage fault handler
    0,                                  // Reserved
    0,                                  // Reserved
    0,                                  // Reserved
    0,                                  // Reserved
    SVC_Handler,                        // SVCall handler
    DebugMon_Handler,                   // Debug monitor handler
    0,                                  // Reserved
    PendSV_Handler,                     // The PendSV handler
    SysTick_Handler,                    // The SysTick handler

    // Chip Level - MK64F12
    DMA0_IRQHandler,                        // 16 : DMA Channel 0 Transfer Complete
    ...
```

Table 3-5. Interrupt vector assignments

Address	Vector	IRQ[1]	NVIC non-IPR register number [2]	NVIC IPR register number [3]	Source module	Source description
ARM Core System Handler Vectors						
0x0000_0000	0	–	–	–	ARM core	Initial Stack Pointer
0x0000_0004	1	–	–	–	ARM core	Initial Program Counter
0x0000_0008	2	–	–	–	ARM core	Non-maskable Interrupt (NMI)
0x0000_000C	3	–	–	–	ARM core	Hard Fault
0x0000_0010	4	–	–	–	ARM core	MemManage Fault
0x0000_0014	5	–	–	–	ARM core	Bus Fault
0x0000_0018	6	–	–	–	ARM core	Usage Fault
0x0000_001C	7	–	–	–	—	—
0x0000_0020	8	–	–	–	—	—
0x0000_0024	9	–	–	–	—	—
0x0000_0028	10	–	–	–	—	—
0x0000_002C	11	–	–	–	ARM core	Supervisor call (SVCall)
0x0000_0030	12	–	–	–	ARM core	Debug Monitor
0x0000_0034	13	–	–	–	—	—
0x0000_0038	14	–	–	–	ARM core	Pendable request for system service (PendableSrvReq)
0x0000_003C	15	–	–	–	ARM core	System tick timer (SysTick)
Non-Core Vectors						
0x0000_0040	16	0	0	0	DMA	DMA channel 0 transfer complete

Kinetis Startup: startup_mk64f12.c

```
//*****************************************************************************
// Functions to carry out the initialization of RW and BSS data sections. These
// are written as separate functions rather than being inlined within the
// ResetISR() function in order to cope with MCUs with multiple banks of
// memory.
//*****************************************************************************
__attribute__ ((section(".after_vectors.init_data")))
void data_init(unsigned int romstart, unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int *pulSrc = (unsigned int*) romstart;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = *pulSrc++;
}

__attribute__ ((section(".after_vectors.init_bss")))
void bss_init(unsigned int start, unsigned int len) {
    unsigned int *pulDest = (unsigned int*) start;
    unsigned int loop;
    for (loop = 0; loop < len; loop = loop + 4)
        *pulDest++ = 0;
}
```

Kinetis Startup: startup_mk64f12.c

```
//*****************************************************************************
// Reset entry point for your code.
// Sets up a simple runtime environment and initializes the C/C++
// library.
//*****************************************************************************
__attribute__ ((section(".after_vectors.reset")))
void ResetISR(void) {
    ...

    //
    // Copy the data sections from flash to SRAM.
    //
    unsigned int LoadAddr, ExeAddr, SectionLen;
    unsigned int *SectionTableAddr;

    // Load base address of Global Section Table
    SectionTableAddr = &__data_section_table;

    // Copy the data sections from flash to SRAM.
    while (SectionTableAddr < &__data_section_table_end) {
        LoadAddr = *SectionTableAddr++;
        ExeAddr = *SectionTableAddr++;
        SectionLen = *SectionTableAddr++;
        data_init(LoadAddr, ExeAddr, SectionLen);
    }
    // At this point, SectionTableAddr = &__bss_section_table;
    // Zero fill the bss segment
    while (SectionTableAddr < &__bss_section_table_end) {
        ExeAddr = *SectionTableAddr++;
        SectionLen = *SectionTableAddr++;
        bss_init(ExeAddr, SectionLen);
    }

    ...

    main();

    //
    // main() shouldn't return, but if it does, we'll just enter an infinite loop
    //
    while (1) {
        ;
    }
}
```

Referencias

Referencias:

- Cortex™-M4 Devices Generic User Guide
- K64 Sub-Family Reference Manual
- Kinetis K64F Sub-Family Data Sheet
- The definitive guide to Cortex-M3/M4
- ARM Developer Documentation
- Manual del GCC, LD, etc